



Floci.io Complete Guide for .NET Developers

With AWS Comparison - Learn Cloud Concepts for FREE

Introduction to Floci.io

What is Floci.io?

Definition: Floci.io ek free, open-source local AWS service emulator hai jo developers ko AWS services ko locally practice karne deta hai bina kisi account, credit, ya complexity ke.

Simple Terms: Floci.io = "AWS ka local practice version" jahan aap free mein sikh sakte ho bina paise invest kiye.

Key Features:

- ✓ FREE - No credit card, no cost
- ✓ LOCAL - Aapke computer par chalata hai
- ✓ COMPLETE - 51 AWS services support karta hai
- ✓ NO SIGNUP - Sirf Docker pull karke start karo
- ✓ FAST - Quick startup, low memory
- ✓ CI READY - Pipeline mein bhi use kar sakte ho

Supported Services (51 Total):

Core Services:	S3, SQS, SNS, DynamoDB, Lambda, IAM, STS
Compute:	EC2, ECS, EKS, Lambda, CodeBuild
Databases:	RDS (PostgreSQL, MySQL), DynamoDB, ElastiCache (Redis)
Messaging:	SQS, SNS, EventBridge, Kinesis
Storage:	S3, ECR
Networking:	API Gateway v1 & v2, Route53, ELB v2
Monitoring:	CloudWatch Logs, CloudWatch Metrics
Security:	KMS, Secrets Manager, ACM, Cognito, IAM
Analytics:	Athena, Glue, Firehose, OpenSearch

Why Learn Floci.io?

Problem: AWS par practice karne ke liye account chahiye, credit chahiye, lekin beginners ko spending risk nahi lena.

Solution with Floci.io:

Without Floci:	With Floci:
- AWS Account needed	- No account needed
- Credit card required	- Completely free
- Real money spend	- Zero cost
- Production data risk	- Safe local testing
- Learning barrier	- Easy start

When to Use Floci.io:

- ✓ Learn AWS concepts first time
- ✓ Practice .NET app with AWS services
- ✓ Build prototypes locally
- ✓ Test CI/CD pipelines
- ✓ Prepare for AWS certification
- ✓ Develop without internet (offline)

When to Move to Real AWS:

- ✓ Production deployment
- ✓ Real users accessing app
- ✓ Need real scalability
- ✓ Compliance requirements
- ✓ Multi-team collaboration
- ✓ Learning production patterns

Floci.io vs AWS vs LocalStack

Aspect	Floci.io	LocalStack	Real AWS
Cost	Free	Free (limited)	Paid
Account	Not required	Not required	Required
Speed	Fast	Medium	N/A
Services	51 services	70+ services	All services
Accuracy	High	Very High	100%
Production Ready	✗ No	✗ No	✓ Yes
Learning	✓ Best	Good	N/A
CI/CD	✓ Yes	✓ Yes	✓ Yes



PART 1: Setting Up Floci.io

Installation Steps

Step 1: Install Docker (If Not Already)

```
# Windows/Mac: Download Docker Desktop from docker.com
# Linux: sudo apt install docker.io

# Check Docker is installed
docker --version
# Docker version 24.0.7, build afdd53b
```

Step 2: Pull and Run Floci.io

Method 1: Docker Run

```
# Run Floci.io container
docker run -d \
  --name floci \
  -p 4566:4566 \
  -p 4510-4559:4510-4559 \
  -v /tmp/floci:/app/data \
  floci/floci:latest

# Check if running
docker ps
# Should see floci container running

# Check logs
docker logs -f floci
```

Method 2: Docker Compose (Recommended)

```
# docker-compose.yml
version: '3.8'

services:
  floci:
    image: floci/floci:latest
    container_name: floci
    ports:
      - "4566:4566"
      - "4510-4559:4510-4559"
    volumes:
      - ./floci-data:/app/data
    environment:
      - SERVICES= -s3 -sqs -sns -dynamodb -lambda -rds -redis
    restart: unless-stopped

volumes:
  floci-data:
```

```
# Start Floci.io
docker-compose up -d

# Check status
docker-compose ps

# View logs
docker-compose logs -f
```

Step 3: Install AWS CLI

Windows:

```
# Download and install AWS CLI
# https://awscli.amazonaws.com/AWSCLIV2.msi

# Or using pip
pip install awscli
```

Linux/Mac:

```
# Using installer
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86\_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install

# Or using pip
pip install awscli
```

Step 4: Configure AWS CLI for Floci

```
# Configure AWS CLI with dummy credentials (Floci doesn't need real auth)
aws configure

# Enter these values:
# AWS Access Key ID: test
# AWS Secret Access Key: test
# Default region name: us-east-1
# Default output format: json

# OR use environment variables
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test
export AWS_DEFAULT_REGION=us-east-1

# Set Floci endpoint URL
export AWS_ENDPOINT_URL=http://localhost:4566
```

Step 5: Test Installation

```
# Test S3 (most common service)
aws --endpoint-url http://localhost:4566 s3 mb s3://my-test-bucket

# List buckets
aws --endpoint-url http://localhost:4566 s3 ls

# If works, Floci is ready!
```



PART 2: Services Deep Dive

1. Storage (S3)

What it means:

AWS S3 (Simple Storage Service) ek object storage service hai jahan aap files (images, PDFs, videos, documents) store kar sakte ho. Yeh scalable, durable, aur affordable hai.

Daily Life Example:

```
S3 = Cloud Google Drive / Dropbox
- Aap files upload karte ho
- Files cloud mein store hote hain
- Kahi se bhi access kar sakte ho
- Sharing possible hai
```

How to use in Floci.io:

```
# =====
# S3 COMMANDS IN FLOCI.IO
# =====

# 1. Create Bucket (storage container)
aws --endpoint-url http://localhost:4566 s3 mb s3://my-app-bucket

# 2. List all buckets
aws --endpoint-url http://localhost:4566 s3 ls

# 3. Upload file to bucket
aws --endpoint-url http://localhost:4566 s3 cp myfile.txt s3://my-app-bucket/

# 4. Download file from bucket
aws --endpoint-url http://localhost:4566 s3 cp s3://my-app-bucket/myfile.txt ./downloaded-file.txt

# 5. List files in bucket
aws --endpoint-url http://localhost:4566 s3 ls s3://my-app-bucket/

# 6. List files recursively
aws --endpoint-url http://localhost:4566 s3 ls s3://my-app-bucket/ --recursive

# 7. Delete file from bucket
aws --endpoint-url http://localhost:4566 s3 rm s3://my-app-bucket/myfile.txt

# 8. Delete bucket (must be empty first)
aws --endpoint-url http://localhost:4566 s3 rb s3://my-app-bucket

# 9. Delete bucket with force (delete all files first)
aws --endpoint-url http://localhost:4566 s3 rb s3://my-app-bucket --force

# 10. Sync local folder to bucket
aws --endpoint-url http://localhost:4566 s3 sync ./local-folder s3://my-app-bucket/

# 11. Generate pre-signed URL (temporary access)
aws --endpoint-url http://localhost:4566 s3 presign s3://my-app-bucket/private-file.txt
```

Similar option in AWS:

AWS Service: Amazon S3 (Simple Storage Service)
Pricing: \$0.023 per GB (varies by region)
Max File Size: 5TB per file
Storage Classes: Standard, IA, Glacier, etc.

.NET Developer Use Case:

1. Upload User Profile Image:

```
// AWS SDK for .NET
using Amazon.S3;
using Amazon.S3.Model;

// Upload profile image
var client = new AmazonS3Client(
    new AmazonS3Config
    {
        ServiceURL = "http://localhost:4566" // For Floci
        // ServiceURL = "https://s3.amazonaws.com" // For Real AWS
    });

var uploadRequest = new PutObjectRequest
{
    BucketName = "my-app-bucket",
    Key = $"profiles/{userId}/avatar.jpg",
    FilePath = localFilePath,
    ContentType = "image/jpeg"
};

await client.PutObjectAsync(uploadRequest);
```

2. Upload PDF Report:

```
// Generate and upload PDF report
var reportKey = $"reports/{DateTime.Now:yyyy/MM}/sales-report.pdf";

var request = new PutObjectRequest
{
    BucketName = "my-app-bucket",
    Key = reportKey,
    FilePath = "./sales-report.pdf",
    ContentType = "application/pdf"
};

await client.PutObjectAsync(request);
```

3. Download Invoice:

```
// Get invoice from S3
var getRequest = new GetObjectRequest
{
    BucketName = "my-app-bucket",
    Key = $"invoices/{invoiceId}.pdf"
};

using var response = await client.GetObjectAsync(getRequest);
using var stream = response.ResponseStream;
using var fileStream = File.Create($"invoice-{invoiceId}.pdf");
stream.CopyTo(fileStream);
```

4. Generate Pre-signed URL for Private Files:

```
// Generate temporary download link (valid for 1 hour)
var url = client.GetPreSignedURL(new GetPreSignedUrlRequest
{
    BucketName = "my-app-bucket",
    Key = "private-document.pdf",
    Expires = DateTime.Now.AddHours(1)
});

// Use this URL in frontend to download file
```

Practical Steps in .NET:

1. Install AWS SDK:

```
dotnet add package AWSSDK.S3
```

2. Configure in appsettings.json:

```
{
  "AWS": {
    "ServiceURL": "http://localhost:4566", // Floci URL
    "Region": "us-east-1",
    "BucketName": "my-app-bucket"
  }
}
```

3. Create S3 Service:


```

// Services/S3Service.cs
using Amazon.S3;

public class S3Service
{
    private readonly IAmazonS3 _s3Client;
    private readonly string _bucketName;

    public S3Service(IConfiguration configuration)
    {
        var config = new AmazonS3Config
        {
            ServiceURL = configuration["AWS:ServiceURL"]
        };
        _s3Client = new AmazonS3Client(config);
        _bucketName = configuration["AWS:BucketName"];
    }

    public async Task<string> UploadFileAsync(Stream fileStream, string fileName, string
contentType)
    {
        var key = $"{Guid.NewGuid()}/{fileName}";

        var request = new PutObjectRequest
        {
            BucketName = _bucketName,
            Key = key,
            InputStream = fileStream,
            ContentType = contentType
        };

        await _s3Client.PutObjectAsync(request);
        return key;
    }

    public async Task<Stream> DownloadFileAsync(string key)
    {
        var request = new GetObjectRequest
        {
            BucketName = _bucketName,
            Key = key
        };

        var response = await _s3Client.GetObjectAsync(request);
        return response.ResponseStream;
    }

    public async Task DeleteFileAsync(string key)
    {
        await _s3Client.DeleteObjectAsync(_bucketName, key);
    }
}

```

Common Mistakes:

```
# ❌ Wrong: Forgetting endpoint URL
aws s3 mb s3://my-bucket
# Error: Could not connect to AWS

# ✅ Correct: Always specify endpoint
aws --endpoint-url http://localhost:4566 s3 mb s3://my-bucket

# ❌ Wrong: Uploading to wrong bucket name
# ❌ Wrong: Not setting content type (browser won't display)
# ❌ Wrong: Trying to upload without creating bucket first
```

Real Company Usage:

Companies using S3:

- Netflix: Store video content
- Airbnb: Store listing images
- Dropbox: Store user files
- Netflix: Backup data
- Spotify: Store music files

Practice Task:

```
# Task 1: Create bucket
aws --endpoint-url http://localhost:4566 s3 mb s3://dotnet-app-bucket

# Task 2: Upload a file
echo "Hello from .NET app" > test.txt
aws --endpoint-url http://localhost:4566 s3 cp test.txt s3://dotnet-app-bucket/

# Task 3: List and download
aws --endpoint-url http://localhost:4566 s3 ls s3://dotnet-app-bucket/
aws --endpoint-url http://localhost:4566 s3 cp s3://dotnet-app-bucket/test.txt ./downloaded.txt

# Task 4: Cleanup
aws --endpoint-url http://localhost:4566 s3 rm s3://dotnet-app-bucket/test.txt
aws --endpoint-url http://localhost:4566 s3 rb s3://dotnet-app-bucket
```

2. Database (DynamoDB & RDS)

What it means:

Database in Cloud - AWS mein do types ke databases hain:

- **DynamoDB**: NoSQL document database (fast, scalable, serverless-like)
- **RDS**: Relational database (PostgreSQL, MySQL, SQL Server, Aurora)

Daily Life Example:

DynamoDB = Google Sheets (NoSQL - flexible schema)

- No fixed columns
- Fast reads/writes
- Pay per request

RDS = Excel/Access (SQL - structured data)

- Fixed columns/tables
- Relations between tables
- SQL queries

How to use in Floci.io (DynamoDB):

```
# =====
# DYNAMODB COMMANDS IN FLOCI.IO
# =====

# 1. Create Table
aws --endpoint-url http://localhost:4566 dynamodb create-table \
  --table-name Products \
  --attribute-definitions AttributeName=ProductId,AttributeType=S \
  --key-schema AttributeName=ProductId,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST

# 2. List Tables
aws --endpoint-url http://localhost:4566 dynamodb list-tables

# 3. Describe Table
aws --endpoint-url http://localhost:4566 dynamodb describe-table --table-name Products

# 4. Put Item (Insert Data)
aws --endpoint-url http://localhost:4566 dynamodb put-item \
  --table-name Products \
  --item '{"ProductId": {"S": "P001"}, "Name": {"S": "Laptop"}, "Price": {"N": "999.99"}}'

# 5. Get Item (Read Data)
aws --endpoint-url http://localhost:4566 dynamodb get-item \
  --table-name Products \
  --key '{"ProductId": {"S": "P001"}}'

# 6. Query (Read with condition)
aws --endpoint-url http://localhost:4566 dynamodb query \
  --table-name Products \
  --key-condition-expression "ProductId = :id" \
  --expression-attribute-values '{":id": {"S": "P001"}}'

# 7. Scan (Read all items)
aws --endpoint-url http://localhost:4566 dynamodb scan --table-name Products

# 8. Update Item
aws --endpoint-url http://localhost:4566 dynamodb update-item \
  --table-name Products \
  --key '{"ProductId": {"S": "P001"}}' \
  --update-expression "SET Price = :newPrice" \
  --expression-attribute-values '{":newPrice": {"N": "1299.99"}}'

# 9. Delete Item
aws --endpoint-url http://localhost:4566 dynamodb delete-item \
  --table-name Products \
  --key '{"ProductId": {"S": "P001"}}'

# 10. Delete Table
aws --endpoint-url http://localhost:4566 dynamodb delete-table --table-name Products
```

How to use in Floci.io (RDS - PostgreSQL):

```
# =====
# RDS (PostgreSQL) COMMANDS IN FLOCI.IO
# =====

# Floci uses a proxy for RDS - connect via standard PostgreSQL client

# Connect to PostgreSQL (port 45103 by default)
# Host: localhost
# Port: 45103
# Database: postgres
# User: test
# Password: test

# Or via docker exec (if running in container)
docker exec -it floci psql -h localhost -p 45103 -U test -d postgres

# Run SQL commands
# CREATE TABLE users (id SERIAL PRIMARY KEY, name VARCHAR(100), email VARCHAR(100));
# INSERT INTO users (name, email) VALUES ('John', 'john@example.com');
# SELECT * FROM users;
```

Similar option in AWS:

Database	AWS Service	Use Case	Pricing
NoSQL	DynamoDB	High traffic, flexible schema	\$1.25 per million writes
SQL	RDS PostgreSQL	Relational data, ACID	\$0.017 per hour
SQL	RDS MySQL	WordPress, simple apps	\$0.017 per hour
SQL	Aurora	Enterprise, auto-scaling	\$0.10 per hour
In-memory	ElastiCache (Redis)	Caching, sessions	\$0.017 per hour

.NET Developer Use Case:

1. DynamoDB - Store Product Catalog:

```

// Install SDK
dotnet add package AWSSDK.DynamoDBv2

// Model
public class Product
{
    public string ProductId { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
    public string Category { get; set; }
    public DateTime CreatedAt { get; set; }
}

// DynamoDB Service
public class DynamoDbService
{
    private readonly IAmazonDynamoDB _client;
    private readonly string _tableName = "Products";

    public DynamoDbService()
    {
        var config = new AmazonDynamoDBConfig
        {
            ServiceURL = "http://localhost:4566"
        };
        _client = new AmazonDynamoDBClient(config);
    }

    public async Task SaveProductAsync(Product product)
    {
        var item = new Dictionary<string, AttributeValue>
        {
            ["ProductId"] = new AttributeValue { S = product.ProductId },
            ["Name"] = new AttributeValue { S = product.Name },
            ["Price"] = new AttributeValue { N = product.Price.ToString() },
            ["Category"] = new AttributeValue { S = product.Category },
            ["CreatedAt"] = new AttributeValue { S = product.CreatedAt.ToString("0") }
        };

        await _client.PutItemAsync(new PutItemRequest
        {
            TableName = _tableName,
            Item = item
        });
    }

    public async Task<Product> GetProductAsync(string productId)
    {
        var response = await _client.GetItemAsync(new GetItemRequest
        {
            TableName = _tableName,
            Key = new Dictionary<string, AttributeValue>
            {
                ["ProductId"] = new AttributeValue { S = productId }
            }
        });

        if (response.Item == null) return null;

        return new Product
        {
            ProductId = response.Item["ProductId"].S,
            Name = response.Item["Name"].S,
            Price = decimal.Parse(response.Item["Price"].N),

```

```
        Category = response.Item["Category"].S,  
        CreatedAt = DateTime.Parse(response.Item["CreatedAt"].S)  
    };  
}  
}
```

2. RDS PostgreSQL - Store Orders:

```

// Install Npgsql
dotnet add package Npgsql

// Connection string
var connectionString = "Host=localhost;Port=45103;Database=postgres;Username=test;Password=test";

public class OrderRepository
{
    private readonly string _connectionString;

    public OrderRepository()
    {
        // Floci RDS (adjust for real AWS)
        _connectionString =
"Host=localhost;Port=45103;Database=postgres;Username=test;Password=test";

        // Real AWS RDS would be:
        // _connectionString = "Host=mydb.xxx.us-
east-1.rds.amazonaws.com;Port=5432;Database=mydb;Username=admin;Password=secret";
    }

    public async Task<List<Order>> GetOrdersAsync()
    {
        await using var connection = new NpgsqlConnection(_connectionString);
        await connection.OpenAsync();

        await using var command = new NpgsqlCommand("SELECT * FROM orders", connection);
        await using var reader = await command.ExecuteReaderAsync();

        var orders = new List<Order>();
        while (await reader.ReadAsync())
        {
            orders.Add(new Order
            {
                Id = reader.GetInt32(0),
                CustomerName = reader.GetString(1),
                TotalAmount = reader.GetDecimal(2),
                OrderDate = reader.GetDateTime(3)
            });
        }
        return orders;
    }

    public async Task CreateOrderAsync(Order order)
    {
        await using var connection = new NpgsqlConnection(_connectionString);
        await connection.OpenAsync();

        await using var command = new NpgsqlCommand(
            "INSERT INTO orders (customer_name, total_amount, order_date) VALUES (@name, @amount,
@date)",
            connection);
        command.Parameters.AddWithValue("name", order.CustomerName);
        command.Parameters.AddWithValue("amount", order.TotalAmount);
        command.Parameters.AddWithValue("date", order.OrderDate);

        await command.ExecuteNonQueryAsync();
    }
}

```

Common Mistakes:

```
# ❌ Wrong: Trying to connect to DynamoDB without creating table first
# ✅ Create table first, then insert data

# ❌ Wrong: Using wrong key type (must be String, Number, or Binary)
# ✅ Define correct key schema

# ❌ Wrong: Forgetting to set endpoint URL in .NET code
# ✅ Always set ServiceURL in config

# ❌ Wrong: Using SQL syntax in DynamoDB (no JOINS!)
# ✅ DynamoDB is NoSQL - design differently
```

Practice Task:

```
# Task 1: Create DynamoDB table
aws --endpoint-url http://localhost:4566 dynamodb create-table \
  --table-name Users \
  --attribute-definitions AttributeName=UserId,AttributeType=S \
  --key-schema AttributeName=UserId,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST

# Task 2: Insert sample data
aws --endpoint-url http://localhost:4566 dynamodb put-item \
  --table-name Users \
  --item '{"UserId": {"S": "U001"}, "Name": {"S": "John"}, "Email": {"S": "john@example.com"}}'

# Task 3: Query the data
aws --endpoint-url http://localhost:4566 dynamodb get-item \
  --table-name Users \
  --key '{"UserId": {"S": "U001"}}'

# Task 4: Update data
aws --endpoint-url http://localhost:4566 dynamodb update-item \
  --table-name Users \
  --key '{"UserId": {"S": "U001"}}' \
  --update-expression "SET #nm = :newName" \
  --expression-attribute-names '{"#nm": "Name"}' \
  --expression-attribute-values '{":newName": {"S": "John Doe"}}'
```

3. Queue (SQS)

What it means:

SQS (Simple Queue Service) ek managed message queue service hai jo distributed systems mein asynchronous communication enable karti hai.

Daily Life Example:

```
Queue = Restaurant Kitchen Order System
- Orders come in (messages)
- Kitchen processes one by one (workers)
- No order is lost (reliable delivery)
- Orders processed in FIFO order
```


How to use in Floci.io:

```
# =====
# SQS COMMANDS IN FLOCI.IO
# =====

# 1. Create Queue
aws --endpoint-url http://localhost:4566 sqs create-queue \
  --queue-name my-queue \
  --attributes '{"VisibilityTimeout": "30", "MessageRetentionPeriod": "86400"}'

# 2. List Queues
aws --endpoint-url http://localhost:4566 sqs list-queues

# 3. Get Queue URL
aws --endpoint-url http://localhost:4566 sqs get-queue-url --queue-name my-queue

# 4. Send Message
aws --endpoint-url http://localhost:4566 sqs send-message \
  --queue-url http://localhost:4566/000000000000/my-queue \
  --message-body '{"orderId": "12345", "customer": "John", "total": 99.99}'

# 5. Receive Messages
aws --endpoint-url http://localhost:4566 sqs receive-message \
  --queue-url http://localhost:4566/000000000000/my-queue \
  --max-number-of-messages 1 \
  --wait-time-seconds 5

# 6. Delete Message
aws --endpoint-url http://localhost:4566 sqs delete-message \
  --queue-url http://localhost:4566/000000000000/my-queue \
  --receipt-handle "<receipt-handle-value>"

# 7. Purge Queue (delete all messages)
aws --endpoint-url http://localhost:4566 sqs purge-queue \
  --queue-url http://localhost:4566/000000000000/my-queue

# 8. Delete Queue
aws --endpoint-url http://localhost:4566 sqs delete-queue \
  --queue-url http://localhost:4566/000000000000/my-queue
```

Similar option in AWS:

AWS Service: Amazon SQS (Simple Queue Service)
Types:

- Standard Queue: Unlimited throughput, at-least-once delivery
- FIFO Queue: Ordered delivery, exactly-once processing

Pricing: \$0.40 per million requests

.NET Developer Use Case:

1. Background Job Processing:

```

// Install SDK
dotnet add package AWSSDK.SQS

public class OrderQueueService
{
    private readonly IAmazonSQS _sqsClient;
    private readonly string _queueUrl;

    public OrderQueueService()
    {
        var config = new AmazonSQSConfig
        {
            ServiceURL = "http://localhost:4566"
        };
        _sqsClient = new AmazonSQSClient(config);
        _queueUrl = "http://localhost:4566/000000000000/orders";
    }

    // Producer: Send order to queue
    public async Task SendOrderAsync(Order order)
    {
        var message = JsonSerializer.Serialize(order);

        await _sqsClient.SendMessageAsync(new SendMessageRequest
        {
            QueueUrl = _queueUrl,
            MessageBody = message,
            MessageAttributes = new Dictionary<string, MessageAttributeValue>
            {
                ["OrderType"] = new MessageAttributeValue
                {
                    DataType = "String",
                    StringValue = order.Type
                }
            }
        });
    }

    // Consumer: Process orders from queue
    public async Task ProcessOrdersAsync()
    {
        var response = await _sqsClient.ReceiveMessageAsync(new ReceiveMessageRequest
        {
            QueueUrl = _queueUrl,
            MaxNumberOfMessages = 10,
            WaitTimeSeconds = 5
        });

        foreach (var message in response.Messages)
        {
            try
            {
                var order = JsonSerializer.Deserialize<Order>(message.Body);
                await ProcessOrder(order);

                // Delete after successful processing
                await _sqsClient.DeleteMessageAsync(_queueUrl, message.ReceiptHandle);
            }
            catch (Exception ex)
            {
                // Log error, message will become visible again after timeout
                Console.WriteLine($"Error processing order: {ex.Message}");
            }
        }
    }
}

```

```
}  
}
```

2. Email Notification Queue:

```
public async Task QueueEmailAsync(Email email)  
{  
    await _sqsClient.SendMessageAsync(new SendMessageRequest  
    {  
        QueueUrl = _queueUrl,  
        MessageBody = JsonSerializer.Serialize(email),  
        MessageAttributes = new Dictionary<string, MessageAttributeValue>  
        {  
            ["EmailType"] = new MessageAttributeValue  
            {  
                DataType = "String",  
                StringValue = email.Type.ToString()  
            },  
            ["Priority"] = new MessageAttributeValue  
            {  
                DataType = "Number",  
                StringValue = ((int)email.Priority).ToString()  
            }  
        },  
        DelaySeconds = 60 // Delay sending for 1 minute  
    });  
}
```

Common Mistakes:

```
# ❌ Wrong: Forgetting to delete message after processing  
# Result: Message will be processed again (Visibility Timeout)  
  
# ❌ Wrong: Long processing time exceeding Visibility Timeout  
# Result: Another worker picks up same message (duplicate processing)  
  
# ❌ Wrong: Not handling failures properly  
# Result: Messages stuck in queue forever
```

Practice Task:

```
# Task 1: Create email queue  
aws --endpoint-url http://localhost:4566 sqs create-queue --queue-name emails  
  
# Task 2: Get queue URL  
aws --endpoint-url http://localhost:4566 sqs get-queue-url --queue-name emails  
  
# Task 3: Send multiple messages  
for i in {1..5}; do  
    aws --endpoint-url http://localhost:4566 sqs send-message \  
        --queue-url http://localhost:4566/000000000000/emails \  
        --message-body "Email #$i notification"  
done  
  
# Task 4: Receive messages  
aws --endpoint-url http://localhost:4566 sqs receive-message \  
    --queue-url http://localhost:4566/000000000000/emails \  
    --max-number-of-messages 5
```

4. Messaging / Pub-Sub (SNS)

What it means:

SNS (Simple Notification Service) ek publish-subscribe messaging service hai jahan ek message ko multiple subscribers ko bhej sakte ho.

Daily Life Example:

```
SNS = News Channel Alert System
- One channel broadcasts (publisher)
- All TV channels receive (subscribers)
- Multiple devices get same alert
- You can subscribe/unsubscribe anytime
```

How to use in Floci.io:

```
# =====
# SNS COMMANDS IN FLOCI.IO
# =====

# 1. Create Topic
aws --endpoint-url http://localhost:4566 sns create-topic \
    --name my-notifications

# 2. List Topics
aws --endpoint-url http://localhost:4566 sns list-topics

# 3. Create Subscription (Email)
aws --endpoint-url http://localhost:4566 sns subscribe \
    --topic-arn "arn:aws:sns:us-east-1:000000000000:my-notifications" \
    --protocol email \
    --notification-endpoint john@example.com

# 4. Create Subscription (SQS - Queue)
aws --endpoint-url http://localhost:4566 sns subscribe \
    --topic-arn "arn:aws:sns:us-east-1:000000000000:my-notifications" \
    --protocol sqs \
    --notification-endpoint "arn:aws:sqs:us-east-1:000000000000:my-queue"

# 5. List Subscriptions
aws --endpoint-url http://localhost:4566 sns list-subscriptions

# 6. List Subscriptions by Topic
aws --endpoint-url http://localhost:4566 sns list-subscriptions-by-topic \
    --topic-arn "arn:aws:sns:us-east-1:000000000000:my-notifications"

# 7. Publish Message
aws --endpoint-url http://localhost:4566 sns publish \
    --topic-arn "arn:aws:sns:us-east-1:000000000000:my-notifications" \
    --message '{"title": "Order Processed", "body": "Your order #12345 is ready!"}' \
    --subject "Order Update"

# 8. Publish to specific endpoint
aws --endpoint-url http://localhost:4566 sns publish \
    --target-arn "arn:aws:sns:us-east-1:000000000000:my-notifications" \
    --message "Direct message"

# 9. Delete Topic
aws --endpoint-url http://localhost:4566 sns delete-topic \
    --topic-arn "arn:aws:sns:us-east-1:000000000000:my-notifications"
```

Similar option in AWS:

AWS Services:

- SNS (Simple Notification Service): Pub-Sub, push notifications
- EventBridge: Event-driven architecture, rules-based routing
- SES (Simple Email Service): Send emails

Pricing:

- SNS: \$0.50 per million publishes, \$0.02 per 1000 email deliveries
- EventBridge: \$1.00 per million events

.NET Developer Use Case:

1. Multi-Channel Notifications:

```

public class NotificationService
{
    private readonly IAmazonSimpleNotificationServiceClient _snsClient;
    private readonly string _orderTopicArn = "arn:aws:sns:us-east-1:000000000000:order-
notifications";

    public NotificationService()
    {
        var config = new AmazonSimpleNotificationServiceConfig
        {
            ServiceURL = "http://localhost:4566"
        };
        _snsClient = new AmazonSimpleNotificationServiceClient(config);
    }

    public async Task NotifyOrderProcessedAsync(Order order)
    {
        var message = JsonSerializer.Serialize(new
        {
            orderId = order.Id,
            customer = order.CustomerName,
            total = order.TotalAmount,
            timestamp = DateTime.UtcNow
        });

        await _snsClient.PublishAsync(new PublishRequest
        {
            TopicArn = _orderTopicArn,
            Message = message,
            Subject = $"Order #{order.Id} Processed",
            MessageAttributes = new Dictionary<string, MessageAttributeValue>
            {
                ["OrderType"] = new MessageAttributeValue
                {
                    DataType = "String",
                    StringValue = order.Type
                }
            }
        });
    }

    // Subscribe email to notifications
    public async Task SubscribeEmailAsync(string email)
    {
        await _snsClient.SubscribeAsync(new SubscribeRequest
        {
            TopicArn = _orderTopicArn,
            Protocol = "email",
            Endpoint = email
        });
    }
}

```

2. SNS + SQS Fan-out Pattern:

```
// SNS Topic with multiple SQS Queue subscriptions
// This enables:
// - Email notifications via email subscription
// - Order processing via SQS queue subscription
// - Audit logging via another SQS queue

public async Task SetupFanOutAsync()
{
    // Create topic
    var topicArn = (await _snsClient.CreateTopicAsync("order-events")).TopicArn;

    // Subscribe queues
    var queues = new[] { "email-queue", "processing-queue", "audit-queue" };
    foreach (var queue in queues)
    {
        await _snsClient.SubscribeAsync(new SubscribeRequest
        {
            TopicArn = topicArn,
            Protocol = "sqs",
            Endpoint = $"arn:aws:sqs:us-east-1:000000000000:{queue}"
        });
    }
}
```

Practice Task:

```
# Task 1: Create notification topic
aws --endpoint-url http://localhost:4566 sns create-topic --name order-updates

# Task 2: Subscribe to topic
aws --endpoint-url http://localhost:4566 sns subscribe \
  --topic-arn "arn:aws:sns:us-east-1:000000000000:order-updates" \
  --protocol email \
  --notification-endpoint developer@example.com

# Task 3: Publish message
aws --endpoint-url http://localhost:4566 sns publish \
  --topic-arn "arn:aws:sns:us-east-1:000000000000:order-updates" \
  --message "New order received! Order #1001, Amount: $59.99" \
  --subject "New Order Alert"
```

5. Compute (Lambda)

What it means:

Lambda ek serverless compute service hai jahan aap code run kar sakte ho bina servers manage kiye. Sirf code upload karo, Lambda automatically manage karega scaling.

Daily Life Example:

```
Lambda = Vending Machine
- You select item (trigger)
- Machine gives you product (response)
- No employee needed (serverless)
- Pay per item used (pay per invocation)
```

How to use in Floci.io:

```
# =====
# LAMBDA COMMANDS IN FLOCI.IO
# =====

# 1. Create Lambda Function (ZIP package)
# First create a simple Python function
cat > hello-function.py << 'EOF'
def handler(event, context):
    return {
        'statusCode': 200,
        'body': 'Hello from Lambda!'
    }
EOF

# Create ZIP file
zip hello-function.zip hello-function.py

# 2. Create Function
aws --endpoint-url http://localhost:4566 lambda create-function \
    --function-name my-function \
    --runtime python3.9 \
    --handler hello-function.handler \
    --role arn:aws:iam::000000000000:role/test-role \
    --zip-file fileb://hello-function.zip

# 3. List Functions
aws --endpoint-url http://localhost:4566 lambda list-functions

# 4. Invoke Function
aws --endpoint-url http://localhost:4566 lambda invoke \
    --function-name my-function \
    --payload '{"name": "World"}' \
    /tmp/response.json

# View response
cat /tmp/response.json

# 5. Get Function Configuration
aws --endpoint-url http://localhost:4566 lambda get-function-configuration \
    --function-name my-function

# 6. Update Function Code
aws --endpoint-url http://localhost:4566 lambda update-function-code \
    --function-name my-function \
    --zip-file fileb://hello-function.zip

# 7. Delete Function
aws --endpoint-url http://localhost:4566 lambda delete-function \
    --function-name my-function

# 8. List invoke versions
aws --endpoint-url http://localhost:4566 lambda list-versions-by-function \
    --function-name my-function
```


Similar option in AWS:

AWS Service: AWS Lambda
Languages: Node.js, Python, Java, C#, Go, Ruby, PowerShell
Pricing: \$0.20 per million invocations (varies by memory)
Max Duration: 15 minutes (configurable)
Cold Start: ~100-500ms

.NET Developer Use Case:

1. C# Lambda Function:

```
// Lambda Function Handler (.csproj: amazon.lambda.runtime.core)
using Amazon.Lambda.Core;

[assembly:
LambdaSerializer(typeof(Amazon.Lambda.Serialization.SystemTextJson.DefaultLambdaJsonSerializer))]

public class Function
{
    public Response FunctionHandler(Request input, ILambdaContext context)
    {
        context.Logger.LogInformation($"Processing order: {input.OrderId}");

        return new Response
        {
            Message = $"Order {input.OrderId} processed successfully!",
            OrderId = input.OrderId,
            ProcessedAt = DateTime.UtcNow
        };
    }
}

public class Request
{
    public string OrderId { get; set; }
    public string CustomerName { get; set; }
    public decimal Amount { get; set; }
}

public class Response
{
    public string Message { get; set; }
    public string OrderId { get; set; }
    public DateTime ProcessedAt { get; set; }
}
```

2. Lambda Triggered by SQS:

```
// Lambda triggered when message arrives in SQS queue
public class SqsLambdaFunction
{
    public async Task ProcessMessageAsync(SQSEvent evnt, ILambdaContext context)
    {
        foreach (var record in evnt.Records)
        {
            try
            {
                var order = JsonSerializer.Deserialize<Order>(record.Body);
                context.Logger.LogInformation($"Processing order {order.Id}");

                await _orderService.ProcessAsync(order);
            }
            catch (Exception ex)
            {
                context.Logger.LogError($"Error processing message: {ex.Message}");
                throw; // Lambda will retry
            }
        }
    }
}
```

3. API Gateway Trigger:

```
// Lambda as API endpoint (via API Gateway)
public class ApiLambdaFunction
{
    public async Task<APIGatewayProxyResponse> HandleRequestAsync(
        APIGatewayProxyRequest request,
        ILambdaContext context)
    {
        if (request.HttpMethod == "GET")
        {
            var products = await _productService.GetAllAsync();
            return new APIGatewayProxyResponse
            {
                StatusCode = 200,
                Body = JsonSerializer.Serialize(products),
                Headers = new Dictionary<string, string> { { "Content-Type", "application/
json" } }
            };
        }

        if (request.HttpMethod == "POST")
        {
            var product = JsonSerializer.Deserialize<Product>(request.Body);
            await _productService.CreateAsync(product);
            return new APIGatewayProxyResponse
            {
                StatusCode = 201,
                Body = JsonSerializer.Serialize(product),
                Headers = new Dictionary<string, string> { { "Content-Type", "application/
json" } }
            };
        }

        return new APIGatewayProxyResponse
        {
            StatusCode = 400,
            Body = "Unsupported operation"
        };
    }
}
```

Practice Task:

```
# Task 1: Create simple Python Lambda
cat > my-function.py << 'EOF'
import json

def handler(event, context):
    body = {
        'message': 'Hello from Lambda!',
        'input': event
    }
    return {
        'statusCode': 200,
        'body': json.dumps(body)
    }
EOF

zip my-function.zip my-function.py

aws --endpoint-url http://localhost:4566 lambda create-function \
    --function-name hello-lambda \
    --runtime python3.9 \
    --handler my-function.handler \
    --role arn:aws:iam::000000000000:role/test-role \
    --zip-file fileb://my-function.zip

# Task 2: Invoke Lambda
aws --endpoint-url http://localhost:4566 lambda invoke \
    --function-name hello-lambda \
    --payload '{"name": "Developer"}' \
    /tmp/lambda-response.json

cat /tmp/lambda-response.json

# Task 3: Cleanup
aws --endpoint-url http://localhost:4566 lambda delete-function --function-name hello-lambda
```

6. Container Registry (ECR)

What it means:

ECR (Elastic Container Registry) ek managed Docker container registry service hai jahan aap Docker images store aur manage kar sakte ho.

Daily Life Example:

```
ECR = Private Docker Hub for your company
- Store your custom Docker images
- Version control your images
- Access control (who can pull/push)
- Scan images for vulnerabilities
```

How to use in Floci.io:

```
# =====
# ECR COMMANDS IN FLOCI.IO
# =====

# 1. Create Repository
aws --endpoint-url http://localhost:4566 ecr create-repository \
  --repository-name my-dotnet-api \
  --image-scanning-configuration scanOnPush=true \
  --encryption-configuration encryptionType=AES256

# 2. List Repositories
aws --endpoint-url http://localhost:4566 ecr describe-repositories

# 3. Get Repository URI
aws --endpoint-url http://localhost:4566 ecr describe-repositories \
  --repository-names my-dotnet-api

# 4. Authorize Docker (get login command)
aws --endpoint-url http://localhost:4566 ecr get-login-password

# 5. Tag Image for ECR
docker tag myapi:latest \
  localhost:4566/000000000000/my-dotnet-api:latest

# 6. Push Image to Repository
docker push localhost:4566/000000000000/my-dotnet-api:latest

# 7. List Images in Repository
aws --endpoint-url http://localhost:4566 ecr list-images \
  --repository-name my-dotnet-api

# 8. Describe Image
aws --endpoint-url http://localhost:4566 ecr describe-images \
  --repository-name my-dotnet-api \
  --image-ids imageTag=latest

# 9. Delete Image
aws --endpoint-url http://localhost:4566 ecr batch-delete-image \
  --repository-name my-dotnet-api \
  --image-ids imageTag=latest

# 10. Delete Repository
aws --endpoint-url http://localhost:4566 ecr delete-repository \
  --repository-name my-dotnet-api \
  --force
```

Similar option in AWS:

AWS Service: Amazon ECR (Elastic Container Registry)
Types:

- Private Registry: For your organization
- Public Registry: For public images (ECR Public)

Pricing: \$0.10 per GB per month (varies by region)
Features: Vulnerability scanning, image tagging, lifecycle policies

.NET Developer Use Case:

1. Build and Push .NET API Image:

```

# Build .NET API
dotnet new webapi -n MyApi
cd MyApi
dotnet publish -c Release -o ./publish

# Create Dockerfile
# (See Docker course for Dockerfile)

# Build image
docker build -t myapi:v1 .

# Tag for ECR (Floci)
docker tag myapi:v1 \
    localhost:4566/000000000000/my-dotnet-api:v1

# Push to registry
docker push localhost:4566/000000000000/my-dotnet-api:v1

# For real AWS:
# docker tag myapi:v1 \
#     123456789012.dkr.ecr.us-east-1.amazonaws.com/my-dotnet-api:v1
# docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/my-dotnet-api:v1

```

2. Pull Image from ECR in .NET Deployment:

```

// Program.cs - Configure Docker image deployment
// This is typically used in ECS/EKS deployments

var taskDefinition = new Dictionary<string, object>
{
    ["family"] = "myapi-task",
    ["containerDefinitions"] = new[]
    {
        new Dictionary<string, object>
        {
            ["name"] = "myapi",
            ["image"] = "000000000000.dkr.ecr.us-east-1.amazonaws.com/my-dotnet-api:v1",
            ["memory"] = 512,
            ["cpu"] = 256,
            ["essential"] = true,
            ["portMappings"] = new[]
            {
                new Dictionary<string, object>
                {
                    ["containerPort"] = 5000,
                    ["protocol"] = "tcp"
                }
            },
            ["environment"] = new[]
            {
                new Dictionary<string, object>
                {
                    ["name"] = "ASPNETCORE_ENVIRONMENT",
                    ["value"] = "Production"
                }
            }
        }
    }
};

```

Practice Task:

```
# Task 1: Create ECR repository
aws --endpoint-url http://localhost:4566 ecr create-repository \
  --repository-name myapp-api

# Task 2: Build sample .NET image (use hello-world or nginx)
docker pull nginx
docker tag nginx localhost:4566/000000000000/myapp-api:latest

# Task 3: Push to registry
docker push localhost:4566/000000000000/myapp-api:latest

# Task 4: List images
aws --endpoint-url http://localhost:4566 ecr list-images --repository-name myapp-api
```

7. Monitoring and Logs (CloudWatch)

What it means:

CloudWatch AWS ki monitoring service hai jo logs, metrics, aur alerts provide karti hai. Aap application performance, errors, aur resource usage monitor kar sakte ho.

Daily Life Example:

```
CloudWatch = Car Dashboard + GPS Tracker
- Speedometer (metrics - CPU, memory usage)
- Odometer (logs - total requests)
- Warning lights (alerts - errors, threshold exceeded)
- Trip recorder (logs - what happened when)
```

How to use in Floci.io:


```
# =====
# CLOUDWATCH COMMANDS IN FLOCI.IO
# =====

# Logs Commands

# 1. Create Log Group
aws --endpoint-url http://localhost:4566 logs create-log-group \
  --log-group-name /my-app/api

# 2. List Log Groups
aws --endpoint-url http://localhost:4566 logs describe-log-groups

# 3. Create Log Stream
aws --endpoint-url http://localhost:4566 logs create-log-stream \
  --log-group-name /my-app/api \
  --log-stream-name api-server-1

# 4. Put Log Events
aws --endpoint-url http://localhost:4566 logs put-log-events \
  --log-group-name /my-app/api \
  --log-stream-name api-server-1 \
  --log-events ' [{"timestamp": "$(date +%s000)", "message": "Application started"} ]'

# 5. Get Log Events
aws --endpoint-url http://localhost:4566 logs filter-log-events \
  --log-group-name /my-app/api \
  --start-time 1700000000000 \
  --end-time 1800000000000

# 6. Delete Log Group
aws --endpoint-url http://localhost:4566 logs delete-log-group \
  --log-group-name /my-app/api

# Metrics Commands

# 7. Put Metric Data
aws --endpoint-url http://localhost:4566 cloudwatch put-metric-data \
  --namespace MyApp \
  --metric-name RequestCount \
  --value 100 \
  --timestamp 1700000000

# 8. Get Metric Statistics
aws --endpoint-url http://localhost:4566 cloudwatch get-metric-statistics \
  --namespace MyApp \
  --metric-name RequestCount \
  --start-time 1700000000 \
  --end-time 1700100000 \
  --period 300 \
  --statistics Average

# Alarms Commands

# 9. Create Alarm
aws --endpoint-url http://localhost:4566 cloudwatch put-metric-alarm \
  --alarm-name high-cpu \
  --alarm-description "CPU usage too high" \
  --metric-name CPUUtilization \
  --namespace AWS/EC2 \
  --statistic Average \
  --period 300 \
  --threshold 80 \
```

```
--comparison-operator GreaterThanThreshold \  
--evaluation-periods 2
```

Similar option in AWS:

AWS Service: Amazon CloudWatch

Components:

- CloudWatch Logs: Store and search logs
- CloudWatch Metrics: Numerical data (CPU, memory, requests)
- CloudWatch Alarms: Trigger notifications on conditions
- CloudWatch Dashboards: Visualize metrics

Pricing: \$0.50 per GB logs, \$0.30 per metric per month

.NET Developer Use Case:

1. Structured Logging to CloudWatch:

```
// Install Serilog.Sinks.AwsCloudWatch  
dotnet add package Serilog.Sinks.AwsCloudWatch  
  
// Program.cs  
var loggerConfig = new LoggerConfiguration()  
    .MinimumLevel.Information()  
    .WriteTo.AwsCloudWatch(  
        new AwsCloudWatchLogClientOptions  
        {  
            LogGroupName = "/my-dotnet-app/api",  
            Region = Amazon.RegionEndpoint.USEast1,  
            // For Floci: Use local endpoint if needed  
        },  
        new RenderedMessageFormatter());  
  
Log.Logger = loggerConfig.CreateLogger();  
  
try  
{  
    Log.Information("Application starting...");  
  
    var builder = WebApplication.CreateBuilder(args);  
    builder.Host.UseSerilog();  
  
    var app = builder.Build();  
  
    app.UseSerilogRequestLogging();  
  
    app.MapGet("/health", () => Results.Ok());  
  
    app.Run();  
}  
catch (Exception ex)  
{  
    Log.Fatal(ex, "Application crashed!");  
}  
finally  
{  
    Log.CloseAndFlush();  
}
```

2. Custom Metrics:

```

public class MetricsService
{
    private readonly IAmazonCloudWatchLogs _cloudWatch;
    private const string Namespace = "MyDotNetApp";

    public MetricsService()
    {
        var config = new AmazonCloudWatchLogsConfig
        {
            ServiceURL = "http://localhost:4566" // For Floci
        };
        _cloudWatch = new AmazonCloudWatchLogsClient(config);
    }

    public async Task RecordRequestCountAsync(string endpoint, int count)
    {
        await _cloudWatch.PutMetricDataAsync(new PutMetricDataRequest
        {
            Namespace = Namespace,
            MetricData = new List<MetricDatum>
            {
                new MetricDatum
                {
                    MetricName = "RequestCount",
                    Dimensions = new List<Dimension>
                    {
                        new Dimension { Name = "Endpoint", Value = endpoint }
                    },
                    Value = count,
                    Unit = StandardUnit.Count,
                    Timestamp = DateTime.UtcNow
                }
            }
        });
    }

    public async Task RecordResponseTimeAsync(string endpoint, double milliseconds)
    {
        await _cloudWatch.PutMetricDataAsync(new PutMetricDataRequest
        {
            Namespace = Namespace,
            MetricData = new List<MetricDatum>
            {
                new MetricDatum
                {
                    MetricName = "ResponseTime",
                    Dimensions = new List<Dimension>
                    {
                        new Dimension { Name = "Endpoint", Value = endpoint }
                    },
                    Value = milliseconds,
                    Unit = StandardUnit.Milliseconds,
                    Timestamp = DateTime.UtcNow
                }
            }
        });
    }
}

```

3. CloudWatch Insights Query:

```
# Query logs in CloudWatch Logs Insights
# fields @timestamp, @message
# | filter message like /ERROR/
# | sort @timestamp desc
# | limit 100
```

Practice Task:

```
# Task 1: Create log group
aws --endpoint-url http://localhost:4566 logs create-log-group \
  --log-group-name /dotnet-app/production

# Task 2: Create log stream
aws --endpoint-url http://localhost:4566 logs create-log-stream \
  --log-group-name /dotnet-app/production \
  --log-stream-name web-server-1

# Task 3: Put sample log
aws --endpoint-url http://localhost:4566 logs put-log-events \
  --log-group-name /dotnet-app/production \
  --log-stream-name web-server-1 \
  --log-events '["timestamp":'$(date +%s000)', "message": "API request: GET /
products", "allocationId": "i-12345"]]'

# Task 4: Get logs
aws --endpoint-url http://localhost:4566 logs filter-log-events \
  --log-group-name /dotnet-app/production \
  --filter-pattern "ERROR"
```

8. Identity and Access (IAM)

What it means:

IAM (Identity and Access Management) AWS resources ko secure karne ka service hai. Yeh define karta hai ki kaun (users/services) kya kar sakta hai (permissions).

Daily Life Example:

```
IAM = Office Access Card System
- Each employee has ID card (Identity)
- Cards have different access levels (Permissions)
- Some can open all doors (Admin)
- Some can only enter their floor (User)
- Some can only access server room (Service Account)
```

How to use in Floci.io:

```
# =====
# IAM COMMANDS IN FLOCI.IO
# =====

# 1. Create User
aws --endpoint-url http://localhost:4566 iam create-user \
    --user-name api-service-account

# 2. List Users
aws --endpoint-url http://localhost:4566 iam list-users

# 3. Create Access Key
aws --endpoint-url http://localhost:4566 iam create-access-key \
    --user-name api-service-account

# 4. Create Policy
aws --endpoint-url http://localhost:4566 iam create-policy \
    --policy-name my-s3-policy \
    --policy-document '{
        "Version": "2012-10-17",
        "Statement": [
            {
                "Effect": "Allow",
                "Action": ["s3:GetObject", "s3:PutObject"],
                "Resource": "arn:aws:s3::my-bucket/*"
            }
        ]
    }'
```

```
# 5. Attach Policy to User
aws --endpoint-url http://localhost:4566 iam attach-user-policy \
    --user-name api-service-account \
    --policy-arn "arn:aws:iam::000000000000:policy/my-s3-policy"

# 6. Create Role
aws --endpoint-url http://localhost:4566 iam create-role \
    --role-name my-ec2-role \
    --assume-role-policy-document '{
        "Version": "2012-10-17",
        "Statement": [
            {
                "Effect": "Allow",
                "Principal": {"Service": "ec2.amazonaws.com"},
                "Action": "sts:AssumeRole"
            }
        ]
    }'
```

```
# 7. List Roles
aws --endpoint-url http://localhost:4566 iam list-roles

# 8. Get User
aws --endpoint-url http://localhost:4566 iam get-user \
    --user-name api-service-account

# 9. List Attached Policies
aws --endpoint-url http://localhost:4566 iam list-attached-user-policies \
    --user-name api-service-account

# 10. Delete User
aws --endpoint-url http://localhost:4566 iam delete-user \
    --user-name api-service-account
```

Similar option in AWS:

AWS Service: Amazon IAM

Components:

- Users: Human users (with login credentials)
- Groups: Collection of users with same permissions
- Roles: Temporary permissions for services
- Policies: JSON documents defining permissions

Pricing: Free (for IAM itself)

.NET Developer Use Case:

1. Service Account for S3 Access:

```
// Create service-specific credentials
// This is typically done via AWS Console or CLI

// .NET code using IAM role (for EC2, ECS, Lambda)
var s3Client = new AmazonS3Client();

// When running on EC2 with IAM role, credentials are auto-discovered
// No need to set access key/secret explicitly

// For Floci/local testing, use dummy credentials
var config = new AmazonS3Config
{
    ServiceURL = "http://localhost:4566"
};
var credentials = new BasicAWSCredentials("test", "test");
var s3Client = new AmazonS3Client(credentials, config);
```

2. Principle of Least Privilege - Create Read-Only Policy:

```
// Programmatic policy creation
var policyDocument = @"{
  ""Version"": ""2012-10-17"",
  ""Statement"": [
    {
      ""Effect"": ""Allow"",
      ""Action"": [
        ""s3:GetObject"",
        ""s3:ListBucket""
      ],
      ""Resource"": [
        ""arn:aws:s3:::my-bucket"",
        ""arn:aws:s3:::my-bucket/*""
      ]
    },
    {
      ""Effect"": ""Allow"",
      ""Action"": [
        ""dynamodb:GetItem"",
        ""dynamodb:Query""
      ],
      ""Resource"": ""arn:aws:dynamodb:us-east-1:000000000000:table/Products""
    }
  ]
}";

await _iamClient.CreatePolicyAsync(new CreatePolicyRequest
{
    PolicyName = "ReadOnlyAccess",
    PolicyDocument = policyDocument
});
```

3. Temporary Credentials via STS:

```
// Using STS to get temporary credentials
public class StsService
{
    public async Task<Credentials> GetTemporaryCredentialsAsync()
    {
        var config = new AmazonSTSTConfig
        {
            ServiceURL = "http://localhost:4566"
        };
        var client = new AmazonSecurityTokenServiceClient(config);

        var response = await client.AssumeRoleAsync(new AssumeRoleRequest
        {
            RoleArn = "arn:aws:iam::000000000000:role/my-service-role",
            RoleSessionName = "my-session"
        });

        return response.Credentials;
    }
}
```


Practice Task:

```
# Task 1: Create service user
aws --endpoint-url http://localhost:4566 iam create-user \
  --user-name product-api-user

# Task 2: Create S3 access policy
aws --endpoint-url http://localhost:4566 iam create-policy \
  --policy-name s3-read-write-policy \
  --policy-document '{
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
        "Resource": "arn:aws:s3:::my-app-bucket/*"
      }
    ]
  }'

# Task 3: Attach policy
aws --endpoint-url http://localhost:4566 iam attach-user-policy \
  --user-name product-api-user \
  --policy-arn "arn:aws:iam::000000000000:policy/s3-read-write-policy"

# Task 4: Create access key
aws --endpoint-url http://localhost:4566 iam create-access-key \
  --user-name product-api-user

# Task 5: List users
aws --endpoint-url http://localhost:4566 iam list-users
```

9. Networking

What it means:

Networking in AWS Virtual network infrastructure provide karta hai jo resources ko connect aur secure karta hai.

Daily Life Example:

```
Networking = Office Building Network Setup
- Floors (VPC - isolated network)
- Rooms (Subnets - network segments)
- Doors (Security Groups - access control)
- Reception (Load Balancer - traffic distribution)
- Intercom (Private endpoints - internal communication)
```

How to use in Floci.io:

```
# =====
# NETWORKING COMMANDS IN FLOCI.IO
# =====

# API Gateway (REST API)
aws --endpoint-url http://localhost:4566 apigateway create-rest-api \
  --name "my-api"

aws --endpoint-url http://localhost:4566 apigateway get-rest-apis

# API Gateway v2 (HTTP API / WebSocket)
aws --endpoint-url http://localhost:4566 apigatewayv2 create-api \
  --name "my-http-api" \
  --protocol-type HTTP

# Route53 (DNS)
aws --endpoint-url http://localhost:4566 route53 create-hosted-zone \
  --name "myapp.example.com" \
  --caller-reference "my-unique-id"

aws --endpoint-url http://localhost:4566 route53 list-hosted-zones

# Route53 create record
aws --endpoint-url http://localhost:4566 route53 change-resource-record-sets \
  --hosted-zone-id "/hostedzone/Z1234567890" \
  --change-batch '{
    "Changes": [{
      "Action": "CREATE",
      "ResourceRecordSet": {
        "Name": "api.myapp.example.com",
        "Type": "A",
        "TTL": 300,
        "ResourceRecords": [{"Value": "192.0.2.1"}]
      }
    }]
  }'
```

```
# Security Groups (via EC2)
aws --endpoint-url http://localhost:4566 ec2 create-security-group \
  --group-name my-sg \
  --description "My security group"

aws --endpoint-url http://localhost:4566 ec2 authorize-security-group-ingress \
  --group-name my-sg \
  --protocol tcp \
  --port 5000 \
  --cidr 0.0.0.0/0

# ELB v2 (Load Balancer)
aws --endpoint-url http://localhost:4566 elbv2 create-load-balancer \
  --name my-alb \
  --subnets subnet-12345 subnet-67890 \
  --scheme internet-facing
```

Similar option in AWS:

Networking Service	AWS Service	Use Case
API Gateway	API Gateway REST/HTTP	Create REST APIs

Networking Service	AWS Service	Use Case
DNS	Route53	Domain management
Load Balancer	ELB v2 (ALB/NLB)	Traffic distribution
Security Groups	EC2 Security Groups	Firewall rules
Private Link	VPC Endpoints	Private connectivity

.NET Developer Use Case:

1. API Gateway with Lambda Integration:

```
# Create API Gateway with Lambda proxy
# API Gateway -> Lambda -> Your .NET API

aws --endpoint-url http://localhost:4566 apigateway create-rest-api \
  --name "product-api"

# Get API ID from response
API_ID="abc123"

# Create resource
aws --endpoint-url http://localhost:4566 apigateway create-resource \
  --rest-api-id $API_ID \
  --parent-id $(aws --endpoint-url http://localhost:4566 apigateway get-resources --rest-api-id $API_ID --query 'items[0].id' --output text) \
  --path-part products

# Create method (GET)
aws --endpoint-url http://localhost:4566 apigateway put-method \
  --rest-api-id $API_ID \
  --resource-id <resource-id> \
  --http-method GET \
  --authorization-type NONE

# Create Lambda integration
aws --endpoint-url http://localhost:4566 apigateway put-integration \
  --rest-api-id $API_ID \
  --resource-id <resource-id> \
  --http-method GET \
  --type AWS_PROXY \
  --integration-http-method POST \
  --uri "arn:aws:apigateway:us-east-1:lambda:path/2015-03-31/functions/arn:aws:lambda:us-east-1:000000000000:function:my-function/invocations"

# Deploy API
aws --endpoint-url http://localhost:4566 apigateway create-deployment \
  --rest-api-id $API_ID \
  --stage-name prod
```

2. ECS Service with Load Balancer:

```
# docker-compose.yml with networking
version: '3.8'

services:
  api:
    image: myapi:latest
    ports:
      - "5000:5000"
    environment:
      - ASPNETCORE_ENVIRONMENT=Production
    networks:
      - app-net
    deploy:
      replicas: 2
    resources:
      limits:
        memory: 512M

networks:
  app-net:
    driver: bridge
```

Practice Task:

```
# Task 1: Create API Gateway
aws --endpoint-url http://localhost:4566 apigateway create-rest-api \
  --name "dotnet-api"

# Task 2: List APIs
aws --endpoint-url http://localhost:4566 apigateway get-rest-apis

# Task 3: Create Route53 hosted zone
aws --endpoint-url http://localhost:4566 route53 create-hosted-zone \
  --name "myapp.local" \
  --caller-reference "myref123"
```

10. Serverless (Lambda + API Gateway)

What it means:

Serverless means aap code run karte ho bina servers manage kiye. AWS Lambda functions ko automatically scale karta hai, aap sirf pay per invocation.

Daily Life Example:

```
Serverless = Cloud Kitchen (Swiggy/Zomato)
- You order food (trigger Lambda)
- Restaurant prepares (code runs)
- Delivery happens (response)
- You pay per order (pay per invocation)
- No need to hire chef permanently
```

Similar option in AWS:

AWS Lambda: Serverless compute
AWS API Gateway: Serverless API
AWS DynamoDB: Serverless database
AWS S3: Serverless storage
AWS SNS/SQS: Serverless messaging

Cost: Pay only for what you use

- Lambda: \$0.20 per million invocations
- API Gateway: \$3.50 per million requests (HTTP API)

.NET Developer Use Case:

1. Full Serverless API:

```
// ProductApi.cs - Lambda Handler
public class Function
{
    private readonly IProductService _productService;

    public Function()
    {
        _productService = new ProductService();
    }

    public async Task<APIGatewayProxyResponse> GetProducts(
        APIGatewayProxyRequest request,
        ILambdaContext context)
    {
        var products = await _productService.GetAllAsync();

        return new APIGatewayProxyResponse
        {
            StatusCode = 200,
            Body = JsonSerializer.Serialize(products),
            Headers = new Dictionary<string, string>
            {
                { "Content-Type", "application/json" },
                { "Access-Control-Allow-Origin", "*" }
            }
        };
    }

    public async Task<APIGatewayProxyResponse> GetProduct(
        APIGatewayProxyRequest request,
        ILambdaContext context)
    {
        var id = request.PathParameters["id"];
        var product = await _productService.GetByIdAsync(id);

        if (product == null)
            return new APIGatewayProxyResponse
            {
                StatusCode = 404,
                Body = "Product not found"
            };

        return new APIGatewayProxyResponse
        {
            StatusCode = 200,
            Body = JsonSerializer.Serialize(product),
            Headers = new Dictionary<string, string>
            {
                { "Content-Type", "application/json" }
            }
        };
    }

    public async Task<APIGatewayProxyResponse> CreateProduct(
        APIGatewayProxyRequest request,
        ILambdaContext context)
    {
        var product = JsonSerializer.Deserialize<Product>(request.Body);
        await _productService.CreateAsync(product);

        return new APIGatewayProxyResponse
        {
            StatusCode = 201,
            Body = JsonSerializer.Serialize(product),
        };
    }
}
```

```

        Headers = new Dictionary<string, string>
        {
            { "Content-Type", "application/json" }
        };
    };
}
}

```

2. Event-Driven Architecture:

```

// Lambda triggered by S3 upload
public class S3TriggerFunction
{
    public async Task<string> HandleS3Event(S3Event evnt, ILambdaContext context)
    {
        foreach (var record in evnt.Records)
        {
            var bucket = record.S3.Bucket.Name;
            var key = record.S3.Object.Key;

            context.Logger.LogInformation($"Processing file: {key} from bucket: {bucket}");

            // Process the uploaded file
            await ProcessFileAsync(bucket, key);
        }

        return "Processed successfully";
    }
}

// Lambda triggered by DynamoDB stream
public class DynamoDbTriggerFunction
{
    public async Task HandleDynamoDbEvent(DynamoDBEvent evnt, ILambdaContext context)
    {
        foreach (var record in evnt.Records)
        {
            if (record.EventName == OperationType.INSERT)
            {
                var product = GetProductFromStream(record);
                await IndexProductAsync(product);
                context.Logger.LogInformation($"Indexed product: {product.Id}");
            }
        }
    }
}

```

Practice Task:

```
# Task 1: Create Lambda function
cat > product-handler.py << 'EOF'
import json

def handler(event, context):
    return {
        'statusCode': 200,
        'body': json.dumps([
            {'id': '1', 'name': 'Product 1', 'price': 99.99},
            {'id': '2', 'name': 'Product 2', 'price': 149.99}
        ])
    }
EOF

zip product-handler.zip product-handler.py

aws --endpoint-url http://localhost:4566 lambda create-function \
    --function-name get-products \
    --runtime python3.9 \
    --handler product-handler.handler \
    --role arn:aws:iam::000000000000:role/test-role \
    --zip-file fileb://product-handler.zip

# Task 2: Invoke function
aws --endpoint-url http://localhost:4566 lambda invoke \
    --function-name get-products \
    --payload '{}' \
    /tmp/response.json

cat /tmp/response.json

# Task 3: Create API Gateway
aws --endpoint-url http://localhost:4566 apigatewayv2 create-api \
    --name "products-api" \
    --protocol-type HTTP

# Task 4: Create Lambda permission for API Gateway
aws --endpoint-url http://localhost:4566 lambda add-permission \
    --function-name get-products \
    --statement-id apigateway-test \
    --action lambda:InvokeFunction \
    --principal apigateway.amazonaws.com
```

11. Secrets Management

What it means:

Secrets Manager sensitive information (passwords, API keys, tokens) ko secure tarike se store aur manage karne ka service hai.

Daily Life Example:

Secrets Manager = Secure Key Locker

- Store passwords (database credentials)
- Store API keys (third-party services)
- Store certificates (SSL/TLS)
- Rotate keys automatically
- Access control (who can see what)

How to use in Floci.io:

```
# =====
# SECRETS MANAGER COMMANDS IN FLOCI.IO
# =====

# 1. Create Secret
aws --endpoint-url http://localhost:4566 secretsmanager create-secret \
  --name my-database-password \
  --secret-string "YourStrongPassword123!"

# 2. List Secrets
aws --endpoint-url http://localhost:4566 secretsmanager list-secrets

# 3. Get Secret Value
aws --endpoint-url http://localhost:4566 secretsmanager get-secret-value \
  --secret-id my-database-password

# 4. Update Secret
aws --endpoint-url http://localhost:4566 secretsmanager put-secret-value \
  --secret-id my-database-password \
  --secret-string "NewStrongPassword456!"

# 5. Delete Secret
aws --endpoint-url http://localhost:4566 secretsmanager delete-secret \
  --secret-id my-database-password
```

Similar option in AWS:

AWS Services:

- Secrets Manager: Full secrets management with rotation
- Systems Manager Parameter Store: Simple configuration storage

Pricing:

- Secrets Manager: \$0.40 per secret per month
- Parameter Store: Free (standard), \$0.05 per advanced param per month

.NET Developer Use Case:

```
public class SecretsService
{
    private readonly IAmazonSecretsManager _secretsManager;

    public SecretsService()
    {
        var config = new AmazonSecretsManagerConfig
        {
            ServiceURL = "http://localhost:4566"
        };
        _secretsManager = new AmazonSecretsManagerClient(config);
    }

    public async Task<string> GetDatabasePasswordAsync()
    {
        var response = await _secretsManager.GetSecretValueAsync(
            new GetSecretValueRequest { SecretId = "my-database-password" });

        return response.SecretString;
    }

    public async Task<DatabaseCredentials> GetDatabaseCredentialsAsync()
    {
        var response = await _secretsManager.GetSecretValueAsync(
            new GetSecretValueRequest { SecretId = "db-credentials" });

        return JsonSerializer.Deserialize<DatabaseCredentials>(response.SecretString);
    }
}

// In Program.cs
builder.Services.AddScoped<ISecretsService, SecretsService>();

var connectionString = await secretsService.GetDatabasePasswordAsync();
// Use in DbContext
```

Practice Task:

```
# Task 1: Store database password
aws --endpoint-url http://localhost:4566 secretsmanager create-secret \
    --name db-password \
    --secret-string "MySecretPassword123!"

# Task 2: Store API key
aws --endpoint-url http://localhost:4566 secretsmanager create-secret \
    --name payment-api-key \
    --secret-string "sk_live_abcdef123456"

# Task 3: Retrieve secrets
aws --endpoint-url http://localhost:4566 secretsmanager get-secret-value \
    --secret-id db-password

# Task 4: List all secrets
aws --endpoint-url http://localhost:4566 secretsmanager list-secrets
```



PART 3: Complete Comparison Table

Floci.io to AWS Service Mapping

Concept	Floci.io Option	AWS Service	.NET Use Case	Notes
File Storage	S3 (Local)	Amazon S3	Upload PDFs, Images	Floci: Local directory, AWS: Cloud storage
NoSQL Database	DynamoDB (Local)	Amazon DynamoDB	Product catalog, User sessions	Floci: SQLite backend, AWS: Managed service
SQL Database	RDS Proxy (Postgres/MySQL)	Amazon RDS	Orders, Users, Transactions	Floci: Port 45103, AWS: Managed database
Cache	ElastiCache (Redis)	Amazon ElastiCache	Session data, API cache	Floci: Redis emulator, AWS: Managed Redis
Message Queue	SQS (Local)	Amazon SQS	Background jobs, Order processing	Same API, different backend
Pub/Sub Messaging	SNS (Local)	Amazon SNS	Notifications, Alerts	Same API, different backend
Serverless Compute	Lambda (Local)	AWS Lambda	API endpoints, Background tasks	Floci: Python only, AWS: Multi-language
API Gateway	API Gateway (Local)	Amazon API Gateway	REST/HTTP APIs	Same API, different backend
Container Registry	ECR (Local)	Amazon ECR	Store Docker images	Same API, different backend
Logs & Monitoring	CloudWatch (Local)	Amazon CloudWatch	Debug errors, Track metrics	Same API, different backend
Identity & Access	IAM (Local)	Amazon IAM	User management, Permissions	Limited in Floci
Secrets	Secrets Manager (Local)	AWS Secrets Manager	Store passwords, API keys	Same API, different backend
DNS	Route53 (Local)	Amazon Route53	Domain management	Limited in Floci
Load Balancer	ELB v2 (Local)	Amazon ELB/ALB	Traffic distribution	Limited in Floci
Event Bus	EventBridge (Local)	Amazon EventBridge	Event-driven architecture	Same API, different backend
Workflow	Step Functions (Local)	AWS Step Functions	Complex workflows	Limited in Floci
Email	SES (Local)	Amazon SES	Transactional emails	Same API, different backend



PART 4: Complete .NET Integration Guide

Integration Patterns

1. Floci Configuration in .NET

appsettings.json:

```
{
  "AWS": {
    "ServiceURL": "http://localhost:4566",
    "Region": "us-east-1",
    "AccessKey": "test",
    "SecretKey": "test"
  },
  "AWSResources": {
    "S3Bucket": "my-app-bucket",
    "SQSQueue": "orders",
    "SNSTopic": "notifications",
    "DynamoDBTable": "Products",
    "LambdaFunction": "my-function"
  }
}
```

Program.cs:

```
var builder = WebApplication.CreateBuilder(args);

// Add AWS services
builder.Services.AddSingleton<IAmazonS3>(sp =>
{
    var config = new AmazonS3Config
    {
        ServiceURL = builder.Configuration["AWS:ServiceURL"]
    };
    return new AmazonS3Client(
        new BasicAWSCredentials(
            builder.Configuration["AWS:AccessKey"],
            builder.Configuration["AWS:SecretKey"]),
        config);
});

builder.Services.AddSingleton<IAmazonSQS>(sp =>
{
    var config = new AmazonSQSConfig
    {
        ServiceURL = builder.Configuration["AWS:ServiceURL"]
    };
    return new AmazonSQSClient(config);
});

// Add services
builder.Services.AddScoped<IS3Service, S3Service>();
builder.Services.AddScoped<ISqsService, SqsService>();
builder.Services.AddScoped<ISnsService, SnsService>();
builder.Services.AddScoped<IDynamoDbService, DynamoDbService>();
```

2. Environment-Based Configuration

appsettings.Development.json:

```
{
  "AWS": {
    "ServiceURL": "http://localhost:4566",
    "Region": "us-east-1"
  }
}
```

appsettings.Production.json:

```
{
  "AWS": {
    "ServiceURL": "https://s3.amazonaws.com",
    "Region": "us-east-1"
  }
}
```

Switching logic:

```
public class AwsConfiguration
{
    public IAmazonS3 CreateS3Client(IConfiguration configuration)
    {
        var serviceUrl = configuration["AWS:ServiceURL"];

        if (string.IsNullOrEmpty(serviceUrl) || serviceUrl.Contains("localhost"))
        {
            // Use Floci configuration
            return new AmazonS3Client(
                new BasicAWSCredentials("test", "test"),
                new AmazonS3Config { ServiceURL = serviceUrl });
        }
        else
        {
            // Use real AWS (credentials from IAM role or environment)
            return new AmazonS3Client(Amazon.RegionEndpoint.USEast1);
        }
    }
}
```

3. S3 + Lambda + API Gateway Pattern

```
// docker-compose.yml for local development
version: '3.8'

services:
  # Floci (AWS services emulator)
  floci:
    image: floci/floci:latest
    ports:
      - "4566:4566"
      - "4510-4559:4510-4559"
    volumes:
      - ./floci-data:/app/data

  # Your .NET API
  api:
    build: .
    ports:
      - "5000:5000"
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - AWS__ServiceURL=http://floci:4566
      - AWS__Region=us-east-1
    depends_on:
      - floci

networks:
  default:
    name: app-network
```

PART 5: When to Move from Floci.io to Real AWS

Decision Matrix

Scenario	Use Floci.io	Use Real AWS
Learning AWS concepts	✓ Yes	✗ No
Building prototypes	✓ Yes	✗ No
CI/CD testing	✓ Yes	✗ No
Local development	✓ Yes	✗ No
Production deployment	✗ No	✓ Yes
Real users	✗ No	✓ Yes
Compliance requirements	✗ No	✓ Yes
Multi-region deployment	✗ No	✓ Yes
Team collaboration	✗ No	✓ Yes
Production monitoring	✗ No	✓ Yes

Migration Checklist

```
# When moving from Floci to AWS:

# 1. Remove endpoint URL
# ✗ builder.Configuration["AWS:ServiceURL"]
# ✓ No service URL needed (use default)

# 2. Set up real credentials
aws configure
# Enter real AWS credentials

# 3. Update resource ARNs
# ✗ arn:aws:s3:::my-bucket
# ✓ Use actual bucket names

# 4. Set up VPC and security groups
# ✗ Floci has no VPC concept
# ✓ Configure real networking

# 5. Update environment variables
ASPNETCORE_ENVIRONMENT=Production
AWS_Region=us-east-1
# No ServiceURL needed

# 6. Test thoroughly
# - Integration tests
# - Performance tests
# - Security tests
```



PART 6: Summary and Quick Reference

Complete Command Reference

```
# =====
# QUICK REFERENCE - ALL FLOCI.IO COMMANDS
# =====

# Setup
docker run -d --name floci -p 4566:4566 -v /tmp/floci:/app/data floci/floci:latest
export AWS_ENDPOINT_URL=http://localhost:4566
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test

# S3
aws s3 mb s3://my-bucket
aws s3 ls
aws s3 cp file.txt s3://my-bucket/
aws s3 rm s3://my-bucket/file.txt

# DynamoDB
aws dynamodb create-table --table-name Users ...
aws dynamodb put-item --table-name Users ...
aws dynamodb get-item --table-name Users ...
aws dynamodb scan --table-name Users

# SQS
aws sqs create-queue --queue-name my-queue
aws sqs send-message --queue-url ... --message-body "Hello"
aws sqs receive-message --queue-url ...

# SNS
aws sns create-topic --name my-topic
aws sns subscribe --topic-arn ... --protocol email ...
aws sns publish --topic-arn ... --message "Alert"

# Lambda
aws lambda create-function --function-name my-func ...
aws lambda invoke --function-name my-func ...

# ECR
aws ecr create-repository --repository-name my-repo
docker tag image localhost:4566/000000000000/my-repo:latest
docker push localhost:4566/000000000000/my-repo:latest

# CloudWatch
aws logs create-log-group --log-group-name /my-app
aws logs put-log-events --log-group-name /my-app ...
aws cloudwatch put-metric-data --namespace MyApp ...

# IAM
aws iam create-user --user-name my-user
aws iam create-policy --policy-name my-policy
aws iam attach-user-policy --user-name my-user ...

# Secrets Manager
aws secretsmanager create-secret --name my-secret --secret-string "value"
aws secretsmanager get-secret-value --secret-id my-secret
```


Floci.io Limitations

- ⚠ Floci.io does NOT support:
 - ✗ Real multi-region deployment
 - ✗ Real authentication/authorization (all "test" credentials)
 - ✗ Real scalability and performance
 - ✗ Real data durability (local storage only)
 - ✗ Real networking across machines
 - ✗ Complex IAM roleAssumeRole chains
 - ✗ Some advanced Lambda features
 - ✗ Real-time CloudWatch metrics aggregation
- ✓ Floci.io DOES support:
 - ✓ Learning all AWS service APIs
 - ✓ Local development and testing
 - ✓ CI/CD pipeline testing
 - ✓ Prototype building
 - ✓ Syntax and code pattern testing



FINAL SUMMARY

Why Floci.io is Amazing for .NET Developers

- ✓ FREE forever - No credit card, no cost
- ✓ Learn AWS concepts without risk
- ✓ Practice 51 AWS services locally
- ✓ Same API calls work in real AWS
- ✓ Easy Docker setup - one command
- ✓ Perfect for CI/CD pipelines
- ✓ Build complete prototypes locally
- ✓ Interview preparation
- ✓ AWS certification prep

Next Steps After Floci.io

1. ✓ Master all Floci.io services
2. ✓ Build a complete .NET app with AWS services
3. ✓ Deploy to real AWS for production
4. ✓ Learn AWS Best Practices
5. ✓ Get AWS Certified
6. ✓ Implement real-world patterns

Interview Ready Checklist

- ✓ Can explain what S3 is and when to use it
- ✓ Can explain DynamoDB vs RDS differences
- ✓ Can explain SQS vs SNS use cases
- ✓ Can explain Lambda serverless pattern
- ✓ Can explain API Gateway routing
- ✓ Can explain IAM roles and policies
- ✓ Can explain CloudWatch monitoring
- ✓ Can explain microservices patterns with AWS
- ✓ Can write .NET code using AWS SDK
- ✓ Can migrate from Floci to real AWS

Congratulations! 🎉

Aapne Floci.io aur AWS services completely seekh liye hain. Ab aap:

- ✓ Cloud concepts locally practice kar sakte ho
- ✓ .NET applications mein AWS services integrate kar sakte ho
- ✓ Floci.io se real AWS pe migrate kar sakte ho
- ✓ AWS interview ke liye ready ho

Best of luck! 🚀

Floci.io Complete Guide for .NET Developers Hinglish Edition - Cloud Learning for FREE